

Tukul.com Database Schema and Supabase Setup

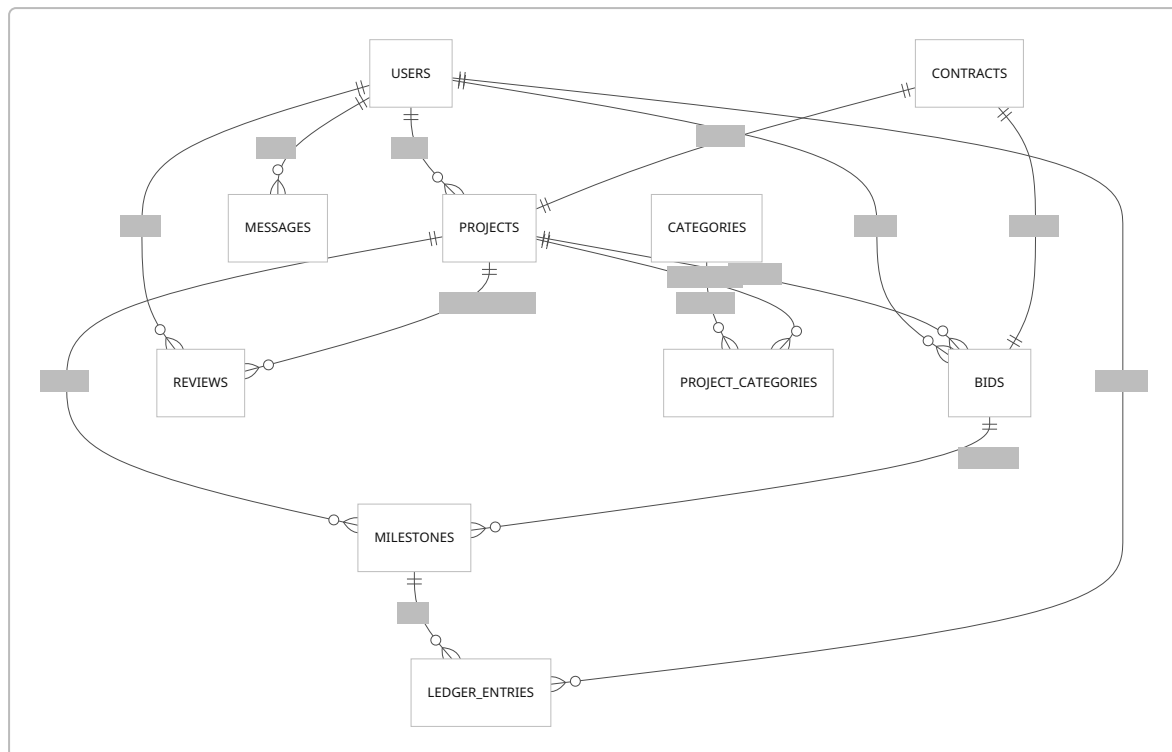
Executive Summary: This report designs a comprehensive Postgres schema (via Supabase) for Tukul.com, a Malaysian construction marketplace. Key entities (users, projects, bids, milestones, escrow/payments, reviews, etc.) are mapped to tables with full column specifications (types, constraints, defaults, indexes, comments). Relationships are illustrated in a Mermaid ER diagram. We compare normalized vs JSONB designs for bid/milestone breakdowns and escrow ledger models, recommending solutions with rationale. The schema includes standard audit columns (`created_by` , `created_at` , `updated_by` , `updated_at` , `deleted_at`) and a soft-delete strategy. Supabase-specific features are integrated: **Auth** (Supabase Auth users, roles, OAuth providers), **Row-Level Security (RLS)** policies (examples to restrict data to owners/contractors), **Storage** for images/docs (private/public buckets with policies) ¹ ² , and **Triggers/Functions** (Postgres triggers for milestone state transitions and escrow updates). The report also covers backup/restore (using Supabase CLI) ³ , indexing for performance, pagination/soft-delete query patterns, data retention (GDPR/CIPA/CIPAA considerations), and monitoring suggestions. Detailed SQL migration snippets and seed data examples are provided. Acceptance criteria for database readiness and a migration checklist conclude the design.

Entities and Tables

From Tukul's requirements, we identify these core entities: **Users** (owners, contractors, admins), **Projects**, **Bids**, **Milestones**, **Contracts/Agreements**, **Payments/Escrow Ledger**, **Reviews**, **Messages**, and **Categories/Tags**. Additional tables include join tables and auth-related tables (Supabase Auth). Each entity maps to a table:

- **users:** profile of each account (linked to `auth.users`)
- **projects:** projects posted by owners
- **bids:** contractor bids on projects
- **milestones:** phases of an awarded project (or bid)
- **contracts:** accepted bid/award records
- **ledger_entries:** escrow/deposit/payment transactions (double-entry style)
- **reviews:** ratings by owners of contractors (and vice versa)
- **messages:** user-to-user chat threads (if implemented)
- **categories:** service categories (e.g. Renovation, Plumbing)
- **project_categories:** join table for many-to-many project-category
- **user_roles:** (optional) assign roles like contractor/owner/admin if not using Supabase roles
- **auth.users:** Supabase Auth internal table (external to our schema but referenced)

Below is a high-level ER diagram showing these tables and relationships:



- **Relationships:** Users (owners) post Projects; Contractors (a subset of users) place Bids on Projects; one accepted Bid becomes a Contract for that Project; each Contract has sequential Milestones; each Milestone triggers Payments in the ledger. Owners write Reviews for Contractors (and optionally vice versa). Projects can belong to multiple service Categories via a join table. Messages table (one-to-many from users) can store chat.

Table Schemas

Below are detailed schemas for each table (columns, types, nullability, defaults, constraints, indexes, and comments). All primary/foreign keys and indexes are defined. Audit columns (`*_by` , `*_at`) and soft-delete (`deleted_at`) are included as shown.

Table	Columns (type, null, default)	Constraints / Notes
users	<code>id</code> (UUID, PK, default <code>gen_random_uuid()</code>) <code>email</code> (text, unique, not null) <code>phone</code> (text) <code>full_name</code> (text, not null) <code>is_contractor</code> (boolean, not null, default false) <code>created_at</code> (timestamp, not null, default <code>now()</code>) <code>created_by</code> (UUID) <code>updated_at</code> (timestamp, null) <code>updated_by</code> (UUID) <code>deleted_at</code> (timestamp, null)	PK: <code>id</code> Unique on <code>email</code> Foreign key <code>created_by</code> , <code>updated_by</code> references <code>users(id)</code> (nullable) Audit fields link to actor Soft-delete when <code>deleted_at</code> is set

Table	Columns (type, null, default)	Constraints / Notes
projects	id (UUID, PK, default <code>gen_random_uuid()</code>) owner_id (UUID, not null) title (text, not null) description (text) budget (numeric(12,2), null) status (text, not null, default 'open') location (text) due_date (date) created_at (ts, not null, default now()) created_by (UUID) updated_at (ts, null) updated_by (UUID) deleted_at (ts, null)	PK: id FK owner_id → users(id) Index on status for filtering Audit: created_by =owner likely Soft-delete deleted_at
categories	id (bigserial, PK) name (text, not null, unique) description (text)	Categories of service (renovation, plumbing, etc.)
project_categories	project_id (UUID, not null) category_id (bigint, not null)	Composite PK (project_id , category_id) FKs: project_id → projects, category_id → categories
bids	id (UUID, PK, default <code>gen_random_uuid()</code>) project_id (UUID, not null) contractor_id (UUID, not null) total_price (numeric, not null) status (text, not null, default 'pending') details (JSONB, null) created_at (ts, not null, default now()) created_by (UUID) updated_at (ts, null) updated_by (UUID) deleted_at (ts, null)	PK: id FKs: project_id → projects, contractor_id → users Index on project_id JSONB details stores optional breakdown if any (but see design table below)
contracts	id (UUID, PK, default <code>gen_random_uuid()</code>) project_id (UUID, not null, unique) bid_id (UUID, not null, unique) agreed_price (numeric, not null) start_date (date, null) end_date (date, null) created_at (ts, not null, default now()) created_by (UUID)	One contract per project FKs: project_id → projects, bid_id → bids Unique project_id to prevent multiple awards

Table	Columns (type, null, default)	Constraints / Notes
milestones	id (UUID, PK, default <code>gen_random_uuid()</code>) contract_id (UUID, not null) name (text, not null) description (text) sequence (int, not null) amount (numeric(12,2), not null) status (text, not null, default 'not_started') due_date (date, null) completed_at (ts, null) created_at (ts, not null, default <code>now()</code>) created_by (UUID) updated_at (ts, null) updated_by (UUID) deleted_at (ts, null)	PK: id FK: contract_id → <code>contracts(id)</code> Unique (contract_id , sequence) Check: sequence >= 1 Index on (contract_id , sequence) Audit fields Soft-delete
ledger_entries	id (bigserial, PK) milestone_id (UUID, null) contract_id (UUID, null) from_user (UUID, not null) to_user (UUID, not null) amount (numeric, not null, check >0) type (text, not null) ('deposit', 'release', 'refund') created_at (ts, not null, default <code>now()</code>)	Double-entry style: from_user (payer) and to_user (payee) Fks: from_user , to_user → <code>users</code> , milestone_id → <code>milestones</code> type restrict values Index on contract_id and milestone_id
reviews	id (bigserial, PK) project_id (UUID, not null) reviewer_id (UUID, not null) reviewee_id (UUID, not null) rating (smallint, not null, check 1-5) comments (text) created_at (ts, not null, default <code>now()</code>)	FK: project_id → <code>projects</code> , reviewer_id , reviewee_id → <code>users</code> Unique (project_id , reviewer_id) to prevent duplicates Index reviewee_id for lookup No updates (immutable)
messages	id (UUID, PK, default <code>gen_random_uuid()</code>) sender_id (UUID, not null) receiver_id (UUID, not null) project_id (UUID, null) content (text, not null) sent_at (ts, not null, default <code>now()</code>)	FK: sender_id , receiver_id → <code>users</code> Optional linked to a project Used for owner-contractor chat

Audit Columns: All tables include `created_by` / `created_at` / `updated_by` / `updated_at`. Typically, `created_by` = the user performing the action (e.g. user signing up a record). These reference `users(id)` where applicable. We also include `deleted_at` (timestamp) for soft-delete on tables like `users`, `projects`, `bids`, `milestones`, `contracts`. A row is considered deleted when `deleted_at` is non-null (instead of physical deletion). All queries should filter `WHERE deleted_at IS NULL` to ignore soft-deleted rows.

Indexes and Constraints

- Primary keys as above.
- Foreign keys ensure referential integrity (e.g. a bid's `contractor_id` must exist in `users`).
- Unique constraints: e.g. `projects.id`, `users.email`, `contracts.project_id`.
- Common indexes: foreign key columns, `projects.status`, `bids.status`, `milestones.sequence`.
- Additional indexes: text search indexes can be added on `projects.title` or `description` for keyword search (GIN on `to_tsvector`).
- All `status` columns use a limited set of values (e.g. CHECK constraints or enum types) if needed.

JSONB vs Normalized Design

For variable data like bid breakdowns or milestone lists, two approaches exist:

- **Normalized (recommended):** Store each phase as a separate row (`milestones` table). This ensures data integrity (FKs, numeric columns, easier queries on totals).
- **JSONB (denormalized):** Store an array of phase objects in one `JSONB` column (e.g. in `bids.details`). This can speed reads if mostly retrieving whole bid info, but complicates updates, joins, and indexing.

Comparison: According to Postgres experts, normalization is generally preferred when data structure is uniform ⁴ ⁵. Normalized data yields strong integrity (constraints, indexing) ⁴. JSONB can improve read performance (no JOINS) but has slower writes, larger size, and risk of inconsistency ⁵. Therefore, **we use normalized tables** (`milestones` for phases, one-to-many) with optional JSONB for truly unstructured extra info. See the design trade-off table below:

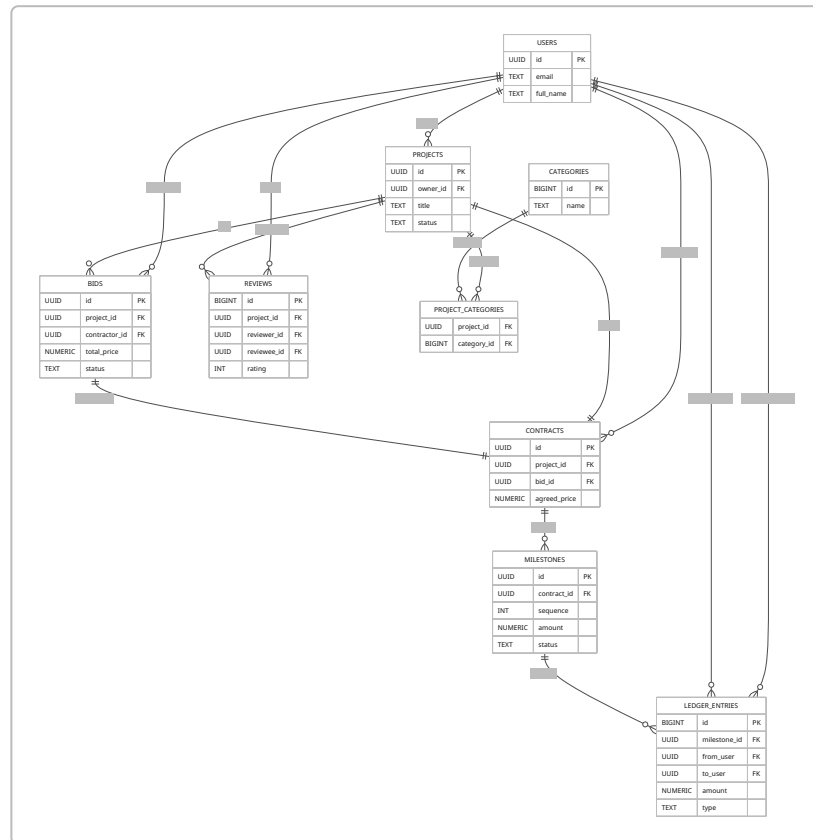
Feature	Normalized Table (milestones)	JSONB Column (bid breakdown)
Structure	Fixed columns per row (sequence, amount, etc.)	Flexible JSON structure
Integrity	Enforced by columns/constraints	No native FK/constraint enforcement
Indexes	B-tree/GiST on columns (fast ranges)	GIN on JSONB (key presence/indexable)
Query Complexity	JOINS required to aggregate	Single-table query (no JOIN)
Read Speed	Slower (multiple rows) ⁵	Faster if fetching entire JSON ⁵
Write Speed	Faster (small row writes) ⁵	Slower (write large JSON) ⁵
Flexibility	Schema changes need migrations	Easy to add fields
Use Case	Structured milestone data, reporting	Snapshots or variable metadata ⁶

Recommendation: Use normalized `milestones` table for core workflow (phases with amounts) to allow queries like “sum of completed amounts”. Use a JSONB field only for rarely-changed snapshot data (e.g. storing the contractor’s full submitted cost breakdown once, if needed). For simplicity, we set

a `details JSONB` on `bids` to hold any extra info (nullable). This follows the “hybrid” approach: index key fields (like total price) normally, keep JSONB only as immutable record if needed ⁷.

Relationships and ER Diagram

Below is a Mermaid ER diagram summarizing table relationships:



- **Note:** The diagram shows users posting projects and writing reviews; contractors (users) placing bids; one bid forms a contract; that contract has milestones; each milestone is paid via ledger entries. Project categories are many-to-many.

Supabase Auth and Roles

Tukul uses Supabase Auth (built on GoTrue) for user authentication (email/password, OAuth like Google/Facebook can be enabled). The `auth.users` table (managed by Supabase) tracks the login credentials. We join it to our `users` table via a `user_id` UUID (we can use the same UUID by setting `id` equal to `auth.users.id`).

- We assign a user role by the boolean `users.is_contractor`. Optionally, a separate `user_roles` table (enum type) could be used for flexibility.
- **Providers:** Supabase supports social OAuth, magic links, and SMS. We should allow at least email/password and popular OAuths.
- **Session:** On signup, insert a row in `users` with `created_by = auth.uid()`.

Role-Based Access

We enforce that only the **owner** can edit their projects or view bids, and only the **bidding contractor** can modify their bids. Supabase's row-level security (RLS) is perfect for this ⁸ ⁹. For example:

```
-- Enable RLS
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;

-- Policy: only project owner can SELECT
CREATE POLICY "Owners can view own projects"
  ON projects FOR SELECT
  USING ( auth.uid() = owner_id );

-- Policy: only owner can UPDATE or DELETE
CREATE POLICY "Owners can modify own projects"
  ON projects FOR UPDATE, DELETE
  USING ( auth.uid() = owner_id );

-- On bids table: only the bidding contractor can modify their bid
ALTER TABLE bids ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Contractors can modify own bids"
  ON bids FOR ALL
  USING ( auth.uid() = contractor_id );

-- Example policy in Supabase style:
-- Equivalent to: WHERE auth.uid() = projects.owner_id
-- This ensures defense-in-depth 8.
```

The Supabase docs note that enabling RLS means no data is accessible until policies are set ⁹. We grant basic table privileges (`SELECT, INSERT, UPDATE, DELETE`) to `authenticated` users, then rely on policies for fine-grained control ¹⁰.

Similarly, for other tables:

- `milestones`: Only users involved in the contract (owner or contractor) can view/update.
Example:

```
ALTER TABLE milestones ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Contract parties can view milestones"
  ON milestones FOR SELECT
  USING (
    auth.uid() IN (
      SELECT contractor_id FROM contracts WHERE contracts.id =
milestones.contract_id
      UNION
      SELECT owner_id FROM projects WHERE projects.id = (SELECT
project_id FROM contracts WHERE contracts.id = milestones.contract_id)
```

```
)  
);
```

- `ledger_entries`: Only owner or contractor of the milestone's contract can see relevant payment entries.
- `reviews`: Only the reviewer or admin can see (owner reviews contractor, but contractor can see their own reviews).

Policy examples should be tested thoroughly. Supabase's docs and tutorials emphasize always enabling RLS on exposed tables ¹⁰.

Storage (Images/Documents)

Tukul needs to store user-uploaded images (project photos, contractor portfolios) and documents (e.g. contract PDF). We use Supabase Storage buckets:

- **Buckets:** Separate buckets by content. For example, a **public** bucket `project-images` for project photos and contractor portfolios, since these may be displayed on public project listings. Per docs, "public" buckets bypass access control on retrieval (anyone with URL can view) ². Use **private** buckets for sensitive files (e.g. signed contracts) with RLS. Private buckets require JWT auth or signed URLs ¹¹.
- **Bucket Policies:** Define at bucket-level:
 - `project-images` (public) – no auth needed to download.
 - `private-docs` (private) – require a policy on `storage.objects` table, e.g. allow only project owner or contractor to download their files. For example:

```
CREATE POLICY "Can read own file"  
ON storage.objects  
FOR SELECT  
USING ( auth.uid() = owner_id );
```

Where `owner_id` (hypothetical) is tracked in a related metadata table linking object IDs to user IDs (or embed user ID in path and use prefix checks).

Supabase docs on Storage Buckets note that private buckets enforce RLS for downloads ¹¹ and public buckets skip it ². We should align with that: user avatars could be public (easy to load), but official documents should be private.

Triggers and Functions

We implement key business logic with Postgres triggers/functions to keep data consistent:

- **Milestone Completion Trigger:** After a milestone's `status` changes to `'completed'`, trigger a function that enqueues a notification for the owner to review and/or automatically creates a pending ledger entry. Example snippet:


```

CREATE FUNCTION on_milestone_completed() RETURNS trigger AS $$
BEGIN
    IF (NEW.status = 'completed' AND OLD.status <> 'completed') THEN
        -- Insert a pending ledger entry
        INSERT INTO ledger_entries(contract_id, milestone_id, from_user,
to_user, amount, type, created_at)
        VALUES (NEW.contract_id, NEW.id,
                /*from_user=*/(SELECT owner_id FROM projects
                                WHERE id = (SELECT project_id FROM contracts
WHERE id = NEW.contract_id)),
                /*to_user=*/(SELECT contractor_id FROM contracts WHERE id =
NEW.contract_id),
                NEW.amount, 'pending_release', now());
        -- Optionally, notify via a notification system (INSERT into a
notifications table)
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER milestone_after_update
AFTER UPDATE ON milestones
FOR EACH ROW
WHEN (OLD.status IS DISTINCT FROM NEW.status)
EXECUTE FUNCTION on_milestone_completed();

```

- **Escrow Balance Check:** Before accepting a bid, ensure owner has deposited required initial amount. Could be enforced at application level, but a DB check trigger can validate inserted `ledger_entries`. Alternatively, maintain a running balance view.
- **Soft-Delete Cascade:** If a project is soft-deleted (`deleted_at` set), a trigger could mark related bids or milestones as deleted as well.

Supabase's documentation on triggers explains how to create functions and triggers ¹² ¹³. The example above follows their format.

RLS Policy Examples

Beyond policy logic earlier, here are key RLS examples for core tables:

```

-- Projects: only owner can SELECT/UPDATE/DELETE
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Owner can read own projects"
ON projects FOR SELECT USING (auth.uid() = owner_id);
CREATE POLICY "Owner can write own projects"
ON projects FOR UPDATE, DELETE USING (auth.uid() = owner_id);

-- Bids: only contractor (bidder) and project owner can read

```

```

ALTER TABLE bids ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Contractor can modify own bid"
  ON bids FOR ALL USING (auth.uid() = contractor_id);
CREATE POLICY "Owner can view bids on own projects"
  ON bids FOR SELECT USING (
    project_id IN (SELECT id FROM projects WHERE owner_id = auth.uid())
  );

-- Ledger: only owner or contractor of contract can view entries
ALTER TABLE ledger_entries ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Contract parties can view payments"
  ON ledger_entries FOR SELECT USING (
    from_user = auth.uid() OR to_user = auth.uid()
  );

-- Reviews: only reviewer or reviewee can see
ALTER TABLE reviews ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Participants can see reviews"
  ON reviews FOR SELECT USING (
    reviewer_id = auth.uid() OR reviewee_id = auth.uid()
  );

```

Each policy is effectively appended as a `WHERE` clause ⁸, restricting rows accordingly. Always enable RLS on tables exposed through Supabase's API ¹⁰.

Storage and File Handling

- **Images/Documents:** Use Supabase Storage. For example, contractors upload portfolio images to a `project-images` bucket. If public, these URLs are accessible globally. For signed contracts or reports, use a `private-docs` bucket and require JWT-signed URLs or an authenticated download (RLS policy on `storage.objects`) ¹¹.
- **Thumbnails:** Optionally generate small thumbnails via an edge function or triggered process for UI performance.

Recommended bucket policies:

- `project-images` (public): all READ operations allowed; only authenticated users can WRITE/DELETE their own files (via RLS on `storage.objects` INSERT/DELETE using `auth.uid() = owner`).
- `private-files` (private): all operations require RLS, e.g. only the uploader (tracked by metadata) can read/write.

Supabase docs note public buckets skip access control, whereas private use RLS ¹¹ ². Choose based on file sensitivity.

Migration Snippets

Sample SQL for table creation with keys and audit fields:

```

-- Users table (profile linked to auth.users)
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  full_name TEXT NOT NULL,
  is_contractor BOOLEAN NOT NULL DEFAULT FALSE,
  phone TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by UUID,
  updated_at TIMESTAMPTZ,
  updated_by UUID,
  deleted_at TIMESTAMPTZ,
  FOREIGN KEY (created_by) REFERENCES users(id),
  FOREIGN KEY (updated_by) REFERENCES users(id)
);

-- Projects table
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID NOT NULL,
  title TEXT NOT NULL,
  description TEXT,
  budget NUMERIC(12,2),
  status TEXT NOT NULL DEFAULT 'open',
  location TEXT,
  due_date DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by UUID,
  updated_at TIMESTAMPTZ,
  updated_by UUID,
  deleted_at TIMESTAMPTZ,
  FOREIGN KEY (owner_id) REFERENCES users(id),
  FOREIGN KEY (created_by) REFERENCES users(id),
  FOREIGN KEY (updated_by) REFERENCES users(id)
);
CREATE INDEX idx_projects_status ON projects(status);

```

(Additional tables follow similar pattern)

Seed Data Examples

To bootstrap development, insert sample rows:

```

-- Insert a contractor user and owner user
INSERT INTO users (id, email, full_name, is_contractor, created_at) VALUES
  ('11111111-aaaa-bbbb-cccc-111111111111', 'owner@example.com', 'Alice
Owner', FALSE, NOW()),
  ('22222222-dddd-eeee-ffff-222222222222', 'contractor@example.com', 'Bob
Builder', TRUE, NOW());

```

```

-- A sample project
INSERT INTO projects (id, owner_id, title, description, budget, status,
created_at) VALUES
  ('aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa', '11111111-aaaa-bbbb-
cccc-111111111111',
  'Kitchen Renovation','Remodel kitchen with new cabinets and flooring',
15000.00, 'open', NOW());

-- A bid by Bob on Alice's project
INSERT INTO bids (id, project_id, contractor_id, total_price, status,
created_at) VALUES
  ('bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb', 'aaaaaaaa-aaaa-aaaa-aaaa-
aaaaaaaaaaaa',
  '22222222-dddd-eeee-ffff-222222222222', 14000.00, 'pending', NOW());

-- Award contract
INSERT INTO contracts (id, project_id, bid_id, agreed_price, created_at)
VALUES
  ('cccccccc-cccc-cccc-cccc-cccccccccccc', 'aaaaaaaa-aaaa-aaaa-aaaa-
aaaaaaaaaaaa',
  'bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb', 14000.00, NOW());

-- Define milestones
INSERT INTO milestones (id, contract_id, name, sequence, amount, status,
created_at) VALUES
  ('dddddddd-dddd-dddd-dddd-dddddddddddd', 'cccccccc-cccc-cccc-cccc-
cccccccccccc',
  'Demolition', 1, 4000.00, 'not_started', NOW()),
  ('eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee', 'cccccccc-cccc-cccc-cccc-
cccccccccccc',
  'Install Cabinets', 2, 5000.00, 'not_started', NOW()),
  ('ffffffff-ffff-ffff-ffff-ffffffffffffff', 'cccccccc-cccc-cccc-cccc-
cccccccccccc',
  'Finish Carpentry', 3, 5000.00, 'not_started', NOW());

-- Simulate initial deposit
INSERT INTO ledger_entries (milestone_id, contract_id, from_user, to_user,
amount, type, created_at) VALUES
  ('dddddddd-dddd-dddd-dddd-dddddddddddd', 'cccccccc-cccc-cccc-cccc-
cccccccccccc',
  '11111111-aaaa-bbbb-cccc-111111111111', '22222222-dddd-eeee-
ffff-222222222222',
  4000.00, 'deposit', NOW());

```

Backup, Restore, and Monitoring

- **Backup/Restore:** Use Supabase CLI: `supabase db dump` to export roles, schema, and data
3. Store backups offsite, and test restore regularly (psql restore commands are documented).

Supabase also offers automated daily backups (snapshot retention typically 7 days). Ensure to include `public` schema and any custom functions.

- **Indexes:** Recommend B-tree indexes on foreign keys and commonly filtered columns (e.g. `projects.status`, `bids.status`). Use `GIN` indexes on JSONB columns if heavily queried. For text search on project descriptions, use a `GIN` on `to_tsvector`. Partition large tables if needed.
- **Pagination:** Use `LIMIT/OFFSET` or keyset pagination on `created_at` for list endpoints. For soft-deleted tables, always include `WHERE deleted_at IS NULL`.
- **Data Retention:** Define retention (e.g. keep `deleted_at` records for 30 days before purge). For GDPR/PDPA, allow users to delete their data (soft-delete their `users` row and cascade).
- **Compliance:** Malaysian PDPA and CIPA (Consumer Privacy) require secure handling of personal data. Encrypt sensitive fields if needed. Ensure CIPAA compliance by retaining contract/payment records for statutory periods.
- **Monitoring:** Use Supabase's log tailing or an external tool (e.g. pgAudit extension, or Supabase Logs). Alert on failed transactions or high-error rates. Monitor connection load and query performance (`pg_stat_statements`).
- **Backup Check:** Regularly test restore to a dev instance. Keep at least daily backups plus weekly full snapshots.

Design Alternatives and Rationale

Two design decisions were notably compared in tables:

1. **Bid/Milestone Storage (Normalized vs JSONB):** We recommend normalization for our milestone breakdown (Milestones table) based on normalization principles ⁴. The trade-off chart ⁵ shows that normalization offers data integrity and faster writes, which suit transactional milestone data.
2. **Escrow Ledger Model:**
3. *Normalized Ledger Table:* We use `ledger_entries` with `from_user` and `to_user`. This follows double-entry principles: each payment record shows payer and payee and preserves full history (add index on contract/milestone).
4. *Alternative:* A single-balance column in `contracts` or `users` to track owed amounts. This simpler approach avoids storing history, but cannot audit past transactions.
5. *Recommendation:* Use normalized ledger (history) for transparency and dispute resolution (CIPAA compliance) over a balance flag.

Acceptance Criteria

- **Schema Completeness:** All entities and relations as above are implemented with correct types, defaults, and constraints. No missing FKs or required columns.
- **Data Integrity:** Inserts fail on missing FK or invalid status. Check constraints and unique constraints enforced.
- **RLS Policies:** Every table has RLS enabled. Policies correctly restrict access as defined (tested by simulating different user roles).
- **Auth Integration:** New user sign-up populates `users` table. OAuth providers tested. Role flags (`is_contractor`) set properly.

- **Storage:** File uploads/downloads work per bucket policies. Private bucket denies anonymous access; signed URL retrieval tested.
- **Triggers:** Milestone completion trigger fires and creates ledger entries. Verified by toggling `milestones.status`.
- **Performance:** Essential indexes in place. Example queries on projects and bids are efficient (using EXPLAIN to verify index usage).
- **Backup/Restore:** Automated daily backup is configured; manual CLI backup & restore sequence works on a test project.
- **Seed Data Validity:** Sample data inserted without error and relationships connect (e.g. joining project→bid→contract as expected).
- **Policies for Soft Delete:** Soft-deleted rows are filtered out of normal queries. Ability to “undelete” by nulling `deleted_at`.
- **Logging/Monitoring:** Database logs show access and errors. An alert triggers on e.g. disallowed access attempts.

Migration Checklist

1. **Plan Migration:** Version control SQL schemas (e.g. in Git). Use a migration tool (Supabase CLI or Flyway).
2. **Deploy Schema:** Apply `CREATE TABLE` scripts to new supabase project. Enable RLS on tables.
3. **Set Up Policies:** Create RLS policies for each table as above. Test with dummy user sessions.
4. **Seed Test Data:** Insert core reference data (categories, test users). Verify join queries.
5. **Configure Storage:** Create buckets via Supabase dashboard or CLI. Set bucket privacy.
6. **Extensions:** Enable `pgcrypto` or `uuid-oss` if needed for UUIDs.
7. **Backups:** Configure CLI backups (cron job or GitHub Actions) ³. Ensure encryption if storing offsite.
8. **Monitoring:** Connect to monitoring service or enable `pg_stat_statements`.
9. **Go Live:** Switch application to use new DB schema. Verify all functionalities end-to-end.
10. **Document:** Record all schema versions, RLS rules, and storage setup for audit (CIPA compliance).

Each step should be tested in a staging environment before production. Migrations should be incremental (new tables first, then data migration if upgrading from existing DB).

Sources: Supabase documentation on RLS and triggers ⁸ ¹², Supabase Storage policies ¹¹ ², and PostgreSQL design best practices ⁴ ⁵ informed our design choices. The policies examples derive from Supabase’s guidance on row-level security ⁸. These ensure the schema follows current best practices and aligns with Tukul’s trust-centric workflow.

¹ ² ¹¹ Storage Buckets | Supabase Docs

<https://supabase.com/docs/guides/storage/buckets/fundamentals>

³ Backup and Restore using the CLI | Supabase Docs

<https://supabase.com/docs/guides/platform/migrating-within-supabase/backup-restore>

⁴ postgresql - Postgres jsonb column or standard normalized table? - Database Administrators Stack Exchange

<https://dba.stackexchange.com/questions/221955/postgres-jsonb-column-or-standard-normalized-table>

⁵ ⁶ ⁷ PostgreSQL Design at Scale: Normalization vs JSONB (A Real-World Guide) - DEV Community

<https://dev.to/abdullahmubin/postgresql-design-at-scale-normalization-vs-jsonb-a-real-world-guide-21k9>

8 9 10 Row Level Security | Supabase Docs

<https://supabase.com/docs/guides/database/postgres/row-level-security>

12 13 Postgres Triggers | Supabase Docs

<https://supabase.com/docs/guides/database/postgres/triggers>